

DIGITAL WALLET AND UPI PAYMENT SYSTEM

Java Object-Oriented Programming Assignment

1. Aim / Objective

To design and implement a Digital Wallet and UPI Payment System in Java using object-oriented programming concepts, simulating wallet creation, money addition, UPI payments, balance checking, transaction validation, and exception handling.

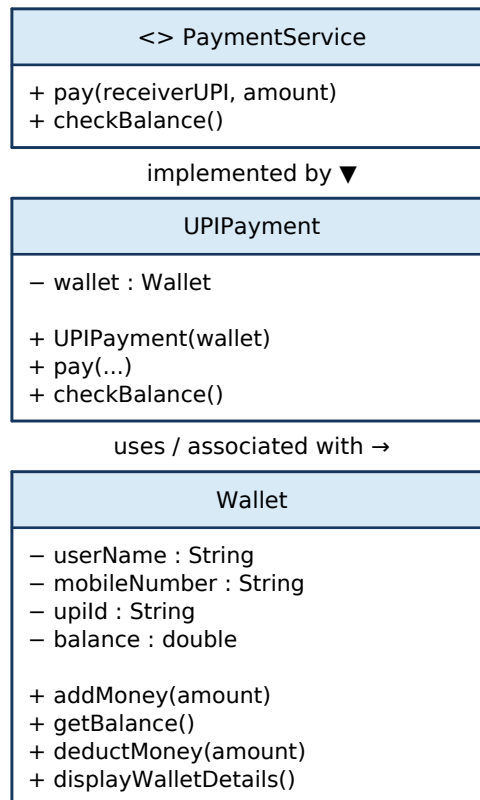
2. Problem Statement

Design a digital wallet for a user. The system should create a wallet with user details, add money, make UPI payments, check balance, validate UPI IDs, prevent invalid or insufficient transactions, handle custom exceptions, and display final wallet details.

3. Concepts Demonstrated

- Interface implementation using PaymentService
- Encapsulation using private data members in Wallet
- Constructors and object creation
- String operations for UPI validation
- Custom exception classes
- try-catch-finally exception handling
- Transaction validation and balance checking

4. UML Class Diagram



Custom Exceptions
InsufficientBalanceException InvalidUPIException InvalidAmountException

PaymentService defines the common payment operations. UPIPayment implements the interface and uses a Wallet object. The three custom exceptions are thrown when validation fails.

5. Algorithm / Logic

1. Create the PaymentService interface and declare pay() and checkBalance().
2. Create the Wallet class with private user details and balance.
3. Define the Wallet constructor and methods addMoney(), getBalance(), and displayWalletDetails().
4. Create custom exceptions for insufficient balance, invalid UPI, and invalid amount.
5. Create UPIPayment implementing PaymentService.
6. Validate the receiver UPI ID using String operations.
7. Validate that the payment amount is greater than zero.
8. Check whether the wallet has sufficient balance.
9. Deduct the amount only when all validations succeed.
10. Handle exceptions using try-catch-finally.
11. Display the final wallet details and balance.

6. Source Code with Comments

```
// Digital Wallet and UPI Payment System
// Demonstrates interface, encapsulation, custom exceptions,
// String validation, and try-catch-finally.

interface PaymentService {
    void pay(String receiverUPI, double amount)
        throws InvalidUPIException, InvalidAmountException,
        InsufficientBalanceException;

    void checkBalance();
}

// Custom exception for insufficient wallet balance
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

// Custom exception for invalid UPI ID
class InvalidUPIException extends Exception {
    public InvalidUPIException(String message) {
        super(message);
    }
}

// Custom exception for invalid amount
class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) {
        super(message);
    }
}

// Wallet class demonstrates encapsulation
class Wallet {
    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    // Constructor to initialize wallet details
    public Wallet(String userName, String mobileNumber,
        String upiId, double balance) {
        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upiId = upiId;
        this.balance = balance;
    }

    // Adds money to the wallet
    public void addMoney(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Money added: Rs." + amount);
        } else {
            System.out.println("Cannot add zero or negative amount.");
        }
    }

    // Returns the current wallet balance
    public double getBalance() {
```

```

        return balance;
    }

    // Deducts money after successful payment
    public void deductMoney(double amount) {
        balance -= amount;
    }

    // Displays wallet details
    public void displayWalletDetails() {
        System.out.println("User Name      : " + userName);
        System.out.println("Mobile Number : " + mobileNumber);
        System.out.println("UPI ID       : " + upiId);
        System.out.printf("Balance        : Rs.%.2f%n", balance);
    }
}

// UPIPayment implements PaymentService
class UPIPayment implements PaymentService {
    private Wallet wallet;

    // Associates a wallet with the payment service
    public UPIPayment(Wallet wallet) {
        this.wallet = wallet;
    }

    // Performs UPI payment after validating all conditions
    @Override
    public void pay(String receiverUPI, double amount)
        throws InvalidUPIException, InvalidAmountException,
        InsufficientBalanceException {

        // Validate UPI ID using String operations
        if (receiverUPI == null || !receiverUPI.contains("@")
            || receiverUPI.startsWith("@")
            || receiverUPI.endsWith("@")) {
            throw new InvalidUPIException("Invalid UPI ID.");
        }

        // Validate payment amount
        if (amount <= 0) {
            throw new InvalidAmountException(
                "Amount must be greater than zero.");
        }

        // Check available balance
        if (amount > wallet.getBalance()) {
            throw new InsufficientBalanceException(
                "Insufficient wallet balance.");
        }

        // Deduct amount after successful validation
        wallet.deductMoney(amount);

        System.out.println("Payment successful!");
        System.out.println("Paid Rs." + amount + " to " + receiverUPI);
    }

    // Displays current balance
    @Override
    public void checkBalance() {
        System.out.printf("Available Balance: Rs.%.2f%n",
            wallet.getBalance());
    }
}

// Main class
public class DigitalWalletSystem {
    public static void main(String[] args) {

        // Create a wallet object
        Wallet wallet = new Wallet(
            "Prasanna", "9876543210",
            "prasanna@upi", 1000);

        // Associate wallet with UPIPayment
        UPIPayment payment = new UPIPayment(wallet);

        System.out.println("==== WALLET DETAILS =====");
        wallet.displayWalletDetails();

        System.out.println("\n==== ADD MONEY =====");
        wallet.addMoney(500);

        System.out.println("\n==== CHECK BALANCE =====");
        payment.checkBalance();

        System.out.println("\n==== UPI PAYMENT =====");

        try {
            payment.pay("ravi@upi", 300);
        } catch (InvalidUPIException | InvalidAmountException |
            InsufficientBalanceException e) {
    
```

```

        System.out.println("Transaction failed: " + e.getMessage());
    } finally {
        System.out.println("Transaction process completed.");
    }

    System.out.println("\n===== INVALID PAYMENT TEST =====");

    try {
        payment.pay("invalidupi", 2000);
    } catch (InvalidUPIException | InvalidAmountException |
            InsufficientBalanceException e) {
        System.out.println("Transaction failed: " + e.getMessage());
    } finally {
        System.out.println("Transaction process completed.");
    }

    System.out.println("\n===== FINAL WALLET DETAILS =====");
    wallet.displayWalletDetails();
    payment.checkBalance();
}
}

```

7. Terminal-Style Commands

8. Output Screen

9. Result

The Digital Wallet and UPI Payment System was implemented successfully in Java. The program creates a wallet, adds money, validates UPI payments, checks the balance, handles invalid transactions using custom exceptions, and displays the final wallet balance.

10. Conclusion

The program demonstrates how interfaces, encapsulation, and exception handling can be used to build a simple and reliable digital payment system.